Your Complete

CLOUD COMPUTING

LEARN.
PRACTICE.
MASTER.
REPEAT.

20 PAGES

COMPLETE ROADMAP + CHEAT SHEET



FROM BASICS TO ADVANCED
All concepts covered step-by-step

REAL-WORLD ARCHITECTURES
Understand through practical examples

TOP 100 INTERVIEW Q&A
Important questions with crisp answers

QUICK REVISION CHEAT SHEET
Key points, formulas & best practices

PERFECT FOR

- Students
- Developers
- SDE Interviews
- Cloud Engineers
- Anyone who wants to grow in Cloud!

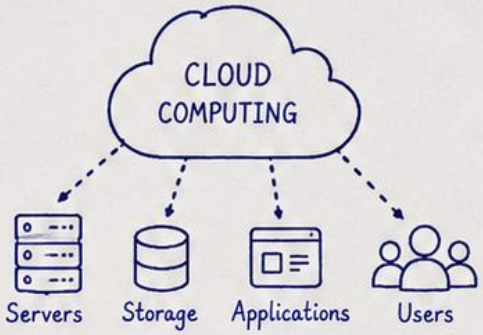
BUILD. DEPLOY. SCALE.
THE CLOUD IS YOURS.



1. Cloud Computing Fundamentals

What is Cloud Computing?

Cloud Computing is the delivery of computing services — including servers, storage, databases, networking, software, analytics, and more — over the Internet ("the cloud") to offer faster innovation, flexible resources, and economies of scale.



Key Characteristics (NIST)

1. On-demand Self-Service — Users can provision resources as needed automatically.
2. Broad Network Access — Services are available over the network via standard mechanisms.
3. Resource Pooling — Provider's resources are pooled to serve multiple consumers.
4. Rapid Elasticity — Resources can scale out and in quickly.
5. Measured Service — Usage is monitored, controlled and reported for transparency.

Benefits of Cloud Computing

- Cost Savings
- Scalability & Flexibility
- High Availability & Reliability
- Faster Time to Market
- Reduced IT Maintenance
- Global Reach

Note :
Pay for what you use.
Scale as you grow.
Focus on your business.

Service Models

IaaS
(Infrastructure as a Service)

Provides virtualized computing resources over the Internet.

Examples: AWS EC2, Azure VM, Google Compute Engine

PaaS
(Platform as a Service)

Provides a platform allowing customers to develop, run and manage applications.

Examples: Google App Engine, Azure App Service, Heroku

SaaS
(Software as a Service)

Delivers software applications over the Internet on subscription.

Examples: Google Workspace, Microsoft 365, Salesforce

Deployment Models

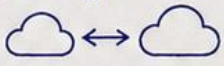
Public Cloud
Resources are owned by the provider and available to the general public.



Private Cloud
Resources are used exclusively by a single organization.



Hybrid Cloud
Combination of public and private cloud with data & app portability.



Community Cloud
Shared by several organizations with common concerns.



Cloud Computing Layers

Application Layer	→ SaaS – End user applications
Platform Layer	→ PaaS – Development platforms
Infrastructure Layer	→ IaaS – Virtual machines, storage, network
Hardware Layer	→ Data centers, servers, storage devices

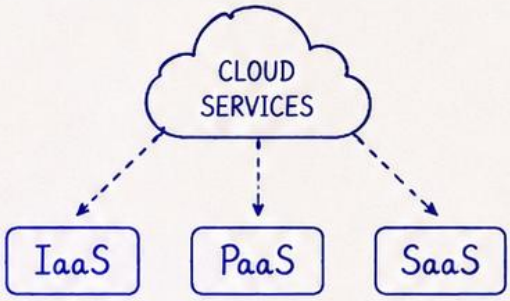
Examples of Cloud Providers

- Amazon Web Services (AWS)
- Microsoft Azure
- Google Cloud Platform (GCP)
- IBM Cloud
- Oracle Cloud

2. Cloud Service Models: IaaS + PaaS + SaaS

What are Cloud Service Models?

Cloud service models define the type of services provided by cloud providers. They determine the level of control, flexibility, management and responsibility you have.



Types of Cloud Service Models

1. IaaS
(Infrastructure as a Service)

- Provides virtualized computing resources over the Internet.
- You manage: OS, storage, applications, runtime, middleware, data.
- Provider manages: hardware, network, virtualization.
- Examples: AWS EC2, Azure VMs, Google Compute Engine

2. PaaS
(Platform as a Service)

- Provides a platform to develop, run, test and deploy applications.
- You manage: applications, data.
- Provider manages: runtime, middleware, OS, servers, storage, networking.
- Examples: Google App Engine, Azure App Service, Heroku

3. SaaS
(Software as a Service)

- Delivers complete software applications over the Internet.
- You don't manage anything.
- Provider manages: everything from hardware to application.
- Examples: Google Workspace, Microsoft 365, Salesforce

Responsibility Comparison

Component	IaaS (You manage)	PaaS (You manage)	SaaS (You manage)
Applications	✓	✓	✗
Data	✓	✓	✗
Runtime	✓	✗	✗
Middleware	✓	✗	✗
Operating System	✓	✗	✗
Virtualization	✗	✗	✗
Servers	✗	✗	✗
Storage	✗	✗	✗
Networking	✗	✗	✗

Legend:
- ✓ Managed by You
- ✗ Managed by Provider

Key Takeaway:
As we move from IaaS → PaaS → SaaS, control decreases but convenience increases.

When to Use?

IaaS
- When you need maximum control over infrastructure.
- When you have specific OS, applications or configurations.
- Example: Hosting custom applications, migrating existing systems.

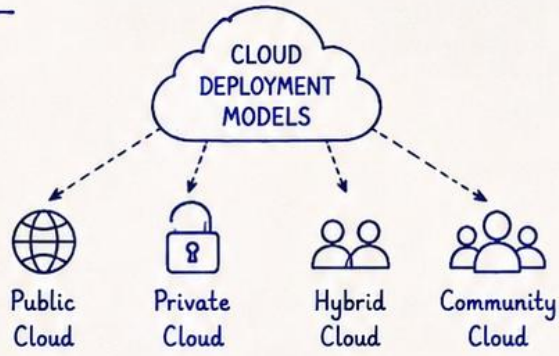
PaaS
- When you want to focus on development, not infrastructure.
- When you need faster development and easy deployment.
- Example: Web applications, APIs, mobile backends.

SaaS
- When you want ready-to-use software over the Internet.
- When you don't want to worry about maintenance.
- Example: Email, CRM, Office tools, collaboration.

3. Cloud Deployment Models

What are Cloud Deployment Models?

Deployment models define how cloud infrastructure is owned, managed and accessed. They specify where the cloud is hosted and who can use it.



Types of Cloud Deployment Models

1. Public Cloud

- Owned and operated by a third-party cloud provider.
- Resources are delivered over the public Internet.
- Highly scalable and cost-effective.
- Examples: AWS, Microsoft Azure, Google Cloud Platform (GCP)

2. Private Cloud

- Used exclusively by a single organization.
- Can be hosted on-premises or by a third-party.
- Greater control, security and customization.
- Examples: VMware vCloud, OpenStack, Microsoft Azure Stack

3. Hybrid Cloud

- Combination of public and private clouds.
- Enables data and application sharing between them.
- Provides flexibility, scalability and cost optimization.
- Example: Using AWS for bursting workloads while keeping sensitive data in private cloud.

4. Community Cloud

- Shared by several organizations with common concerns (e.g., security, compliance, mission).
- Costs and responsibilities are shared.
- More secure and cost-effective than public cloud.
- Example: Government, healthcare organizations.

Comparison

Feature	Public Cloud	Private Cloud	Hybrid Cloud	Community Cloud
Ownership	Third-party	Single Organization	Combination	Several Organizations
Access	Public over Internet	Private Network / Internet	Both Public & Private	Restricted to Community
Control	Low	High	Medium to High	Medium
Security	Standard	High	High	High
Scalability	High	Medium	High	Medium to High
Cost	Pay-as-you-go	High (CapEx)	Optimized	Shared Cost
Best For	Startups, Dev/Test, Dynamic Workloads	Sensitive Data, Compliance, Critical Workloads	Flexibility, Business Continuity	Organizations with Common Goals

When to Use?

- Public Cloud: When you need scalability, quick setup and lower cost.
- Private Cloud: When you need maximum control, security and compliance.
- Hybrid Cloud: When you want flexibility and want to keep critical data private while using public cloud for others.
- Community Cloud: When multiple organizations have similar goals and requirements.

Key Takeaways

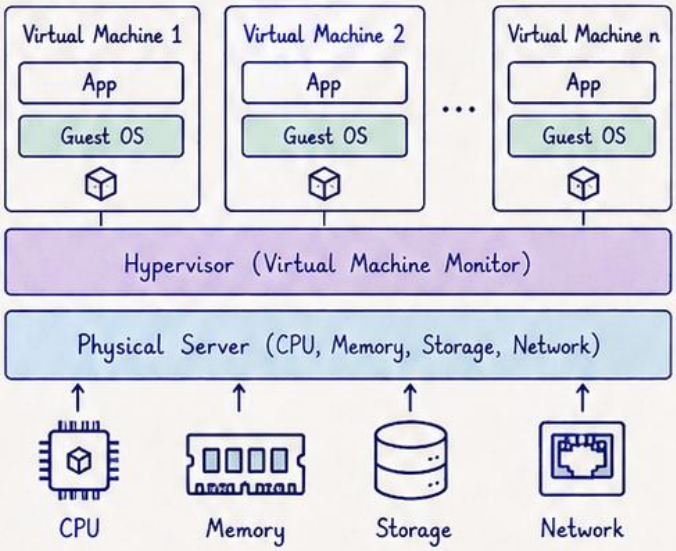
- ☑ Choose the deployment model based on your business needs, security, compliance and budget.
- ☑ Many organizations use Hybrid Cloud for flexibility and optimization.
- ☑ Security, governance and management strategy are critical for all models.

4. Virtualization + Hypervisors

What is Virtualization?

Virtualization is the process of creating a virtual version of something, such as operating systems, servers, storage devices or network resources. It allows multiple virtual instances to run on a single physical machine, improving resource utilization, flexibility and cost efficiency.

How Virtualization Works?



Benefits of Virtualization

- Better resource utilization
- Cost savings
- Scalability & flexibility
- High availability
- Faster provisioning
- Isolation & security
- Disaster recovery
- Easier management
- Reduced physical space
- Green IT (less power)

Hypervisor (Virtual Machine Monitor)

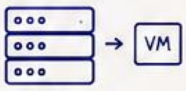
Type	Description	Examples	Characteristics
1. Type 1 (Bare-Metal)	Runs directly on the physical hardware without a host OS.	VMware ESXi, Microsoft Hyper-V, Xen, KVM	• Better performance • More secure • Used in data centers and cloud environments
2. Type 2 (Hosted)	Runs on top of a host operating system.	Oracle VirtualBox, VMware Workstation, Parallels Desktop	• Easier to install • Less performance than Type 1 • Used for testing and development

Note :

Virtualization is a key technology behind cloud computing. It enables cloud providers to offer on-demand resources by running many virtual machines on powerful physical servers.

Types of Virtualization

1. Server Virtualization



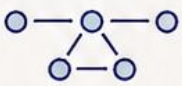
Multiple VMs run on a single physical server.

2. Storage Virtualization



Pools physical storage from multiple devices into a single storage resource.

3. Network Virtualization



Creates virtual networks to improve management and security.

4. Desktop Virtualization



Desktops are hosted centrally and accessed remotely by users.

5. Application Virtualization



Applications run in an isolated environment without installation conflicts.

Use Cases

- ☑ Cloud Infrastructure (IaaS)
- ☑ Consolidation of Servers
- ☑ Dev/Test Environments
- ☑ Business Continuity & DR
- ☑ VDI (Virtual Desktop Infrastructure)
- ☑ Application Isolation

Key Takeaway :

Virtualization abstracts physical resources. Hypervisors make it possible to run more, with less.

5. Cloud Architecture + Components

@codedsoul_05

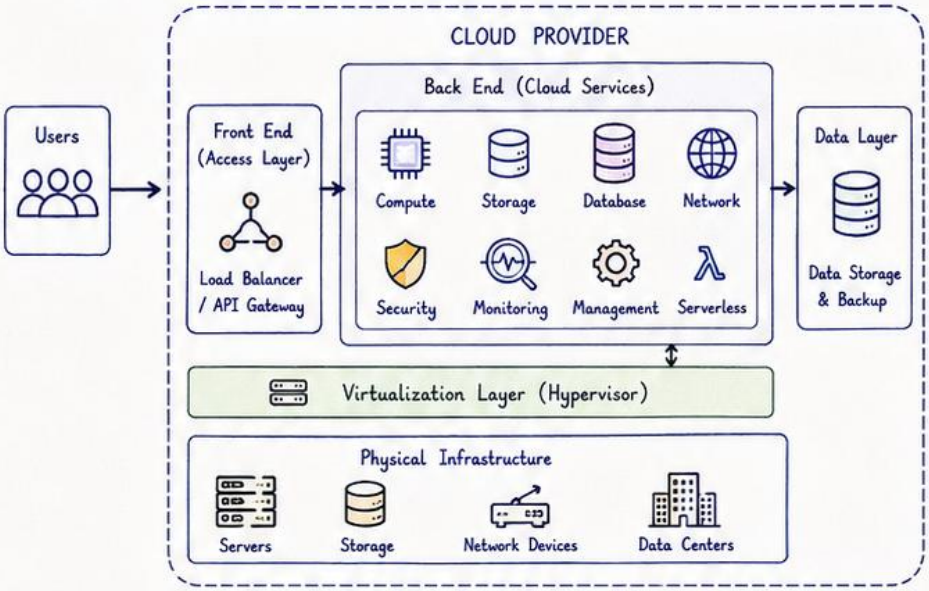
What is Cloud Architecture?

Cloud architecture is the framework of components and services that work together over the internet to deliver computing resources, applications and data on demand with scalability, reliability and security.

Key Principles

- ☑ On-demand Self-Service
- ☑ Broad Network Access
- ☑ Resource Pooling
- ☑ Rapid Elasticity
- ☑ Measured Service
- ☑ High Availability
- ☑ Fault Tolerance
- ☑ Security & Compliance

High Level Cloud Architecture



Core Components

Compute	Provides virtual machines, containers and serverless compute to run applications.
Storage	Offers object, block and file storage for data persistence and backup.
Database	Managed databases (SQL/NoSQL) for scalable and reliable data storage.
Networking	Virtual networks, subnets, routing, DNS, load balancing and CDN.
Security	Identity & Access Management, encryption, firewalls, DDoS protection, compliance.
Management & Monitoring	Monitoring, logging, alerts, automation, orchestration and cost management.
Serverless	Run code without managing servers. Pay only for execution time.

Cloud Architecture Layers

Application Layer	User applications, SaaS, portals, dashboards.
Service Layer (Cloud Services)	Exposes cloud services like compute, storage, database, AI/ML, etc.
Resource Abstraction Layer	Hides physical resources and provides virtualized resources.
Virtualization Layer	Hypervisors create and manage virtual machines/containers.
Physical Layer	Physical servers, storage devices, network hardware, data centers.

Common Cloud Components

- API Gateway : Single entry point for client requests
- Load Balancer : Distributes traffic across resources
- Auto Scaling : Automatically scales resources based on demand
- CDN : Delivers content faster across the globe
- IAM : Manages users, roles and permissions
- Backup & DR : Ensures data protection and recovery
- Monitoring : Tracks performance and health of resources

Benefits of Well-Architected Cloud

Scalability · High Availability · Cost Optimization · Security · Reliability · Performance

Key Takeaway

Cloud architecture is built using modular components that deliver flexibility, scalability, security and efficiency. Designing the right architecture ensures performance, reliability and business success.

Real World Examples

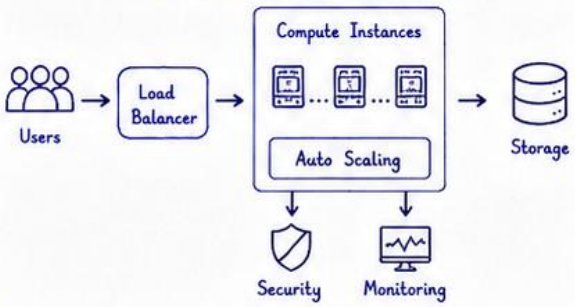
- AWS Architecture (EC2, S3, RDS, VPC, IAM, CloudWatch)
- Azure Architecture (VM, Blob Storage, SQL DB, VNet, AAD, Monitor)
- GCP Architecture (Compute Engine, Cloud Storage, Cloud SQL, VPC, IAM)

6. Compute + Storage Services

What are Compute Services?

Compute services provide virtual machines, containers, and serverless options that let you run applications without managing physical hardware.

Compute Service Flow



Types of Compute Services

1. Virtual Machines



Scalable virtual servers with full control over OS, applications and configurations.

Examples: AWS EC2, Azure VM, GCP Compute Engine

2. Containers



Lightweight, portable and consistent environments to run applications.

Examples: AWS ECS, Azure Container Instances, GCP GKE

3. Serverless



Run code without managing servers. Pay only for actual executions.

Examples: AWS Lambda, Azure Functions, GCP Cloud Functions

4. Managed Compute Services



Fully managed compute services for specific workloads.

Examples: AWS Elastic Beanstalk, Azure App Service

Key Takeaway (Compute)

Choose the right compute service based on your control needs, scalability, cost and application type.

- ☑ Use VMs for full control
- ☑ Use Containers for portability
- ☑ Use Serverless for event-driven & low-maintenance apps
- ☑ Use Managed compute for PaaS-style simplicity

What are Storage Services?

Storage services allow you to store, manage and retrieve data securely, durably and cost-effectively in the cloud.

Types of Storage Services

1. Object Storage



Stores data as objects in buckets. Highly scalable and durable.

Examples: AWS S3, Azure Blob Storage, GCP Cloud Storage

2. Block Storage



Stores data in fixed-size blocks. Used by VMs as virtual hard disks.

Examples: AWS EBS, Azure Managed Disks, GCP Persistent Disk

3. File Storage



Shared file storage accessible via standard file protocols.

Examples: AWS EFS, Azure Files, GCP Filestore

4. Archive Storage



Low-cost storage for long-term data retention and backup.

Examples: AWS Glacier, Azure Archive Storage, GCP Archive Storage

5. Database Storage



Managed database services with automated storage management.

Examples: AWS RDS, Azure SQL Database, GCP Cloud SQL

Key Takeaway (Storage)

Choose storage based on access pattern, performance, durability and cost.

- ☑ Object Storage → for large amounts of unstructured data
- ☑ Block Storage → for VMs
- ☑ File Storage → for shared file systems
- ☑ Archive Storage → for long-term backup
- ☑ Database Storage → for structured data needs

Compute Services Comparison

Service Type	Use Case	Management	Scalability	Cost Model
Virtual Machines	Legacy apps, custom environments	You manage OS, apps, patching	High	Pay per instance (on-demand/RI)
Containers	Microservices, modern applications	Manage containers, not servers	Very High	Pay per container resource usage
Serverless	Event-driven, APIs, background tasks	Fully managed by provider	Auto (on demand)	Pay per request / execution time
Managed Compute	Web apps, business apps, PaaS needs	Platform managed, focus on code	High	Pay per application / instance

Storage Services Comparison

Storage Type	Best For	Durability	Access Speed	Cost
Object Storage	Backup, media, data lakes, static files	Very High	Medium	Low
Block Storage	VMs, databases, transactional apps	High	High	Medium
File Storage	Shared files, enterprise apps	High	Medium	Medium
Archive Storage	Long-term backup, compliance data	Very High	Low	Very Low
Database Storage	Structured data, applications	High	High	Medium to High

Key Points to Remember

- ☆ Compute services help you run your applications.
- ☆ Storage services help you persist and protect your data.
- ☆ Right combination of compute + storage = Scalable, Reliable and Cost-effective solutions.

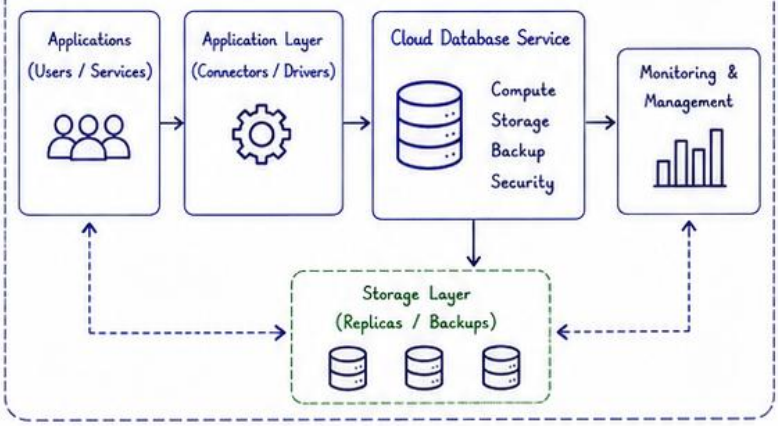
7. Databases in Cloud

@codedsoul_05

What are Databases in Cloud?

Cloud databases are databases that run on a cloud platform. They provide high availability, scalability, security, automated backups and reduced operational overhead compared to traditional on-premise databases.

Cloud Database Architecture



Types of Cloud Databases

1. Relational Databases

- Structured data (tables)
- ACID compliant
- Strong consistency
- Examples: Amazon RDS, Azure SQL Database, Google Cloud SQL

2. NoSQL Databases

- Non-relational, flexible schema
- High scalability & performance
- Types: Key-Value, Document, Column-Family, Graph
- Examples: Amazon DynamoDB, Azure Cosmos DB, MongoDB Atlas

3. Data Warehouse

- Optimized for analytics and reporting
- Handles large volumes of data
- Examples: Amazon Redshift, Google BigQuery, Azure Synapse Analytics

4. In-Memory Databases

- Data stored in memory for ultra-fast performance
- Ideal for real-time apps
- Examples: Amazon ElastiCache, Azure Cache for Redis, Redis Enterprise Cloud

Key Takeaways

- ☑ Cloud databases are fully managed, scalable and highly available.
- ☑ Choose the right database based on your data model, consistency needs and workload type.
- ☑ Use built-in features like backups, encryption, monitoring and auto scaling.
- ☑ Optimize cost by selecting the right instance type and storage model.
- ☑ Always follow best practices for security and access control.

Relational vs NoSQL Databases

Feature	Relational Databases (SQL)	NoSQL Databases
Data Model	Tables (Rows & Columns)	Key-Value / Document / Column / Graph
Schema	Fixed Schema	Flexible Schema
Scalability	Vertical Scaling	Horizontal Scaling
Consistency	Strong Consistency (ACID)	Eventual Consistency (Base)
Use Case	Transactional apps, ERP, CRM	Big Data, Real-time apps, IoT, Content Management
Examples	MySQL, PostgreSQL, SQL Server	DynamoDB, MongoDB, Cosmos DB

Popular Cloud Database Services

Cloud Provider	Relational DB	NoSQL DB	Data Warehouse	In-Memory DB
AWS	Amazon RDS, Aurora	DynamoDB	Redshift	ElastiCache (for Redis)
Microsoft Azure	Azure SQL DB, Azure Database for MySQL	Cosmos DB	Synapse Analytics	Azure Cache for Redis
Google Cloud	Cloud SQL	Firestore, Bigtable	BigQuery	Memorystore (for Redis)

Benefits of Cloud Databases

- **Scalability** — Scale up or down as per demand
- **High Availability** — Built-in replication and failover
- **Automatic Backups** — Point-in-time recovery
- **Reduced Management** — Fully managed by cloud provider
- **Cost Efficiency** — Pay-as-you-go pricing model

Best Practices

- ☆ Choose the right database type for the right workload.
- ☆ Enable encryption at rest and in transit.
- ☆ Use IAM roles and least privilege access.
- ☆ Monitor performance and set up alerts.
- ☆ Regularly backup and test recovery.

Examples of Use Cases

E-commerce
Use relational DB for orders and NoSQL DB for product catalog and sessions.

Mobile Apps
Use NoSQL DB for user profiles, activity feeds and real-time data.

Analytics
Use Data Warehouse for business intelligence and big data analytics.

Gaming
Use Cloud DB for player data, leaderboards and in-game economy.

8. Networking + VPC

@codedsoul_05

What is Networking in Cloud?

Cloud networking enables resources (VMs, databases, storage, containers, etc.) to communicate securely over the internet or private networks. It provides isolation, control, scalability and high availability.

Key Networking Concepts

Internet
Public network accessible from anywhere.

Private Network
Isolated network within the cloud, not directly accessible from internet.

Security Groups
Act as virtual firewall for instances (stateful).

Network ACL
Stateless firewall at subnet level (allow/deny rules).

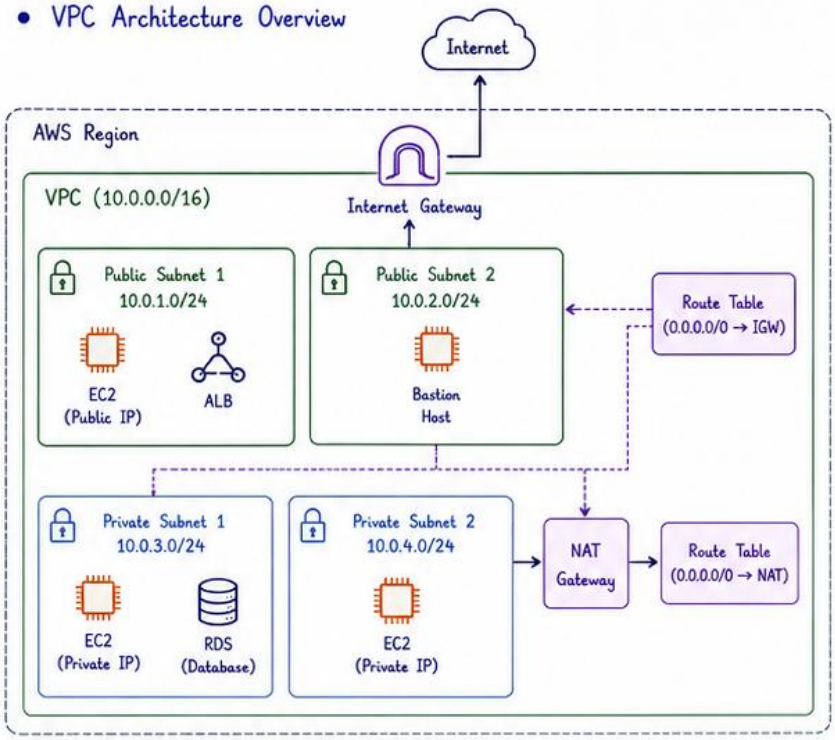
Route Table
Determines where network traffic is directed.

Internet Gateway
Connects your VPC to the internet.

NAT Gateway
Allows instances in private subnet to access internet outbound only.

VPC Peering
Connects two VPCs privately across regions/accounts.

VPC Architecture Overview



VPC Components

Component	Description
VPC	A logically isolated virtual network in the cloud.
Subnet	A range of IP addresses within a VPC (Public or Private).
Internet Gateway	Enables communication between VPC and the internet.
NAT Gateway	Allows private subnet instances to access internet outbound.
Route Table	Contains rules that determine how traffic is routed.
Security Group	Stateful firewall applied to instances (allow rules).
Network ACL	Stateless firewall applied to subnets (allow/deny rules).
VPC Peering	Connects two VPCs using private IP addresses.
Elastic IP	Static public IPv4 address for dynamic cloud resources.

Common Network Traffic Flow

Scenario	Source	Destination	Path
Internet to Public Instance	Internet	EC2 in Public Subnet	Internet → IGW → Route Table → Public Subnet → Instance
Private Instance to Internet (Outbound)	EC2 in Private Subnet	Internet	Instance → NAT Gateway → IGW → Internet
Internet to Private Instance	Internet	EC2 in Private Subnet	Not allowed directly. Use Bastion Host / ALB in Public Subnet.
VPC to VPC (Peering)	VPC A	VPC B	Private IP traffic over VPC Peering Connection

Types of Connectivity

1. Internet Connectivity
- Access resources from internet.
- Use Internet Gateway.

2. Private Connectivity
- Resources communicate within VPC privately.
- No internet exposure.

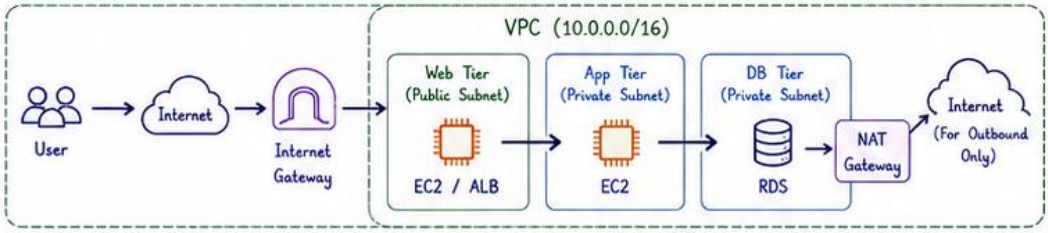
3. Hybrid Connectivity
- Connect on-premise network to VPC.
- Use VPN / Direct Connect.

4. VPC Peering Connectivity
- Connect multiple VPCs.
- Secure and private.

Best Practices

- ☑ Use private subnets for databases and backend services.
- ☑ Use Security Groups with least privilege access.
- ☑ Use NAT Gateway for outbound internet access from private subnets.
- ☑ Enable VPC Flow Logs for monitoring and troubleshooting.
- ☑ Use VPC Peering or Transit Gateway for multi-VPC architectures.
- ☑ Use Network ACLs for an additional layer of security.

Example Diagram: 3-Tier Architecture in VPC



Key Takeaway

VPC gives you complete control over your virtual network in the cloud. By combining subnets, gateways, route tables and security controls, you can build secure, scalable and highly available architectures.

9. Load Balancing + Auto Scaling

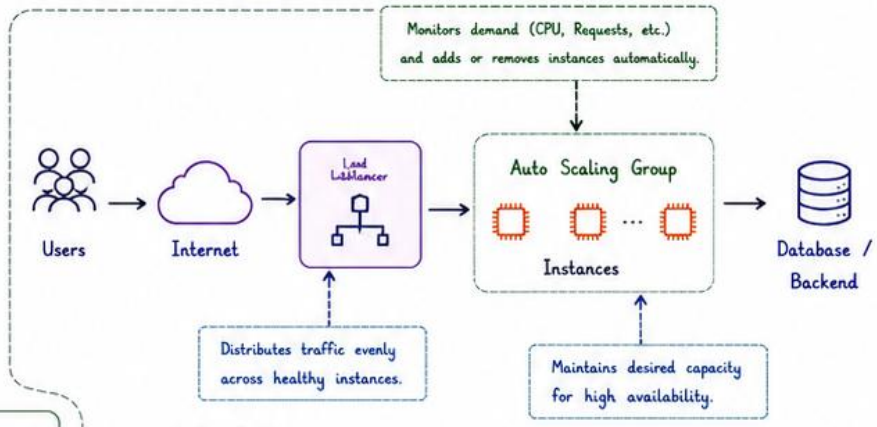
@codedsoul_05

What are Load Balancing and Auto Scaling?

Load Balancing distributes incoming traffic across multiple instances to ensure high availability, improve performance and prevent any single server from becoming a bottleneck.

Auto Scaling automatically adjusts the number of instances based on demand to maintain performance and optimize cost.

How It Works (Together)



Load Balancing

Benefits

- ☑ High availability
- ☑ Improved performance
- ☑ Fault tolerance
- ☑ Scalability
- ☑ SSL termination
- ☑ Health checks

Types

1. Application Load Balancer (L7)
   - HTTP/HTTPS traffic
   - Route based on path, host, headers
2. Network Load Balancer (L4)
   - TCP/UDP traffic
   - Ultra-high performance, static IP
3. Classic Load Balancer
   - Legacy option (use ALB/NLB)

Auto Scaling

Benefits

- ☑ Handles variable load
- ☑ Maintains availability
- ☑ Cost optimization
- ☑ No manual intervention
- ☑ Supports high traffic spikes

Scaling Policies

1. Target Tracking Scaling
   - Scales to maintain a target metric (e.g., CPU = 60%)
2. Step Scaling
   - Scaling based on metric reaching certain thresholds
3. Scheduled Scaling
   - Scale based on time (e.g., office hours)

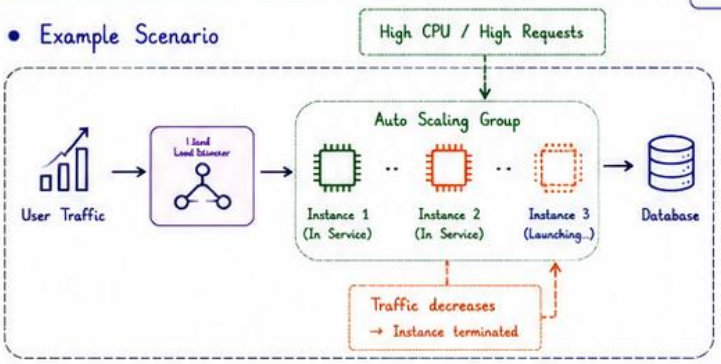
Load Balancer Algorithm

Algorithm	Description
Round Robin	Distributes requests sequentially to each instance.
Least Connections	Sends requests to the instance with the fewest active connections.
IP Hash	Uses client IP to route requests to the same instance.
Least Response Time	Routes to the instance with the lowest response time.
Weighted Round Robin	Distributes based on assigned weight to each instance.

Auto Scaling Components

Component	Description
Launch Template / Configuration	Defines instance details (AMI, type, key, SG, etc.).
Auto Scaling Group (ASG)	Group of instances treated as a logical unit for scaling and management.
Scaling Policies	Rules to scale out or in based on metrics or time.
CloudWatch Alarms	Monitors metrics and triggers scaling actions.
Cooldown Period	Wait time after a scaling activity before next action.

Example Scenario



Scaling Metrics (Examples)

Metric	Used For
CPU Utilization	Most common metric for scaling compute instances.
Request Count (ALB)	Scale based on number of requests.
Network In / Out	Scale based on network traffic.
Custom Metrics	Application specific metrics.

Best Practices

- ☑ Use health checks to route traffic only to healthy instances.
- ☑ Configure proper scaling policies and thresholds.
- ☑ Set minimum, maximum and desired capacity wisely.
- ☑ Use target tracking scaling for most cases.
- ☑ Monitor with CloudWatch and review scaling activities regularly.

Key Takeaway

Load Balancing ensures traffic is distributed efficiently, while Auto Scaling ensures you have the right number of instances at the right time. Together, they provide High Availability, Performance and Cost Efficiency.

10. Cloud Security Fundamentals

@codedsoul_05

What is Cloud Security?

Cloud security refers to the policies, technologies, controls and best practices designed to protect data, applications and infrastructure in the cloud. It ensures Confidentiality, Integrity and Availability of resources.

Key Security Objectives (CIA Triad)

Confidentiality
Prevent unauthorized access to data.

Integrity
Prevent unauthorized modification of data.

Availability
Ensure resources are available when needed.

Shared Responsibility Model

Responsibility	Cloud Provider	Customer
Physical Data Centers	✓	✗
Hardware, Networking, Power	✓	✗
Virtualization Layer	✓	✗
Operating Systems	(Managed)	(You Manage)
Applications	(Managed)	(You Manage)
Data	✗	✓
Identity & Access Management	✗	✓
Security Configuration	✗	✓

✓ = Responsible ✗ = Not Responsible

Cloud Security Controls

Preventive Controls	Stop attacks from happening. Example: IAM, Security Groups, Firewalls
Detective Controls	Detect suspicious activities. Example: Monitoring, Logging, IDS/IPS
Corrective Controls	Respond and fix issues. Example: Patch Management, Backups
Directive Controls	Define security policies and guidelines. Example: Policies, Standards, Compliance

Common Security Threats in Cloud

- Data Breaches
- Weak IAM / Credential Theft
- Misconfiguration
- Malware & Ransomware
- DDoS Attacks
- Insider Threats

Key Takeaway

Security in the cloud is a shared responsibility. Use the right controls, follow best practices and continuously monitor to build a secure cloud environment.

Cloud Security Layers

Data Security	Encryption, DLP, Backup, Data Masking
Application Security	Secure SDLC, Input Validation, WAF, Dependency Scanning
Platform / OS Security	Patching, Hardening, Antivirus, Host Firewall
Network Security	VPC, Subnets, Security Groups, NACL, VPN, IDS/IPS
Infrastructure Security	Physical Security, Hardware, Availability Zones, Redundancy

Identity & Access Management (IAM)

Users → Authenticate (MFA) → Authorize (Policies) → Access Resources

Best Practices

- ☑ Use least privilege access
- ☑ Enable Multi-Factor Authentication (MFA)
- ☑ Rotate and manage credentials securely
- ☑ Use roles instead of long-term access keys

Data Protection

Encryption	• Encrypt data at rest and in transit • Use strong encryption (AES-256, TLS 1.2+)
Key Management	• Use KMS / Key Vault • Rotate keys regularly
Backup & Recovery	• Automated backups • Test recovery processes
Data Governance	• Classification, Retention Policies • Compliance (GDPR, HIPAA, ISO 27001)

Security Best Practices

- ☑ Enable MFA for all users
- ☑ Follow the principle of least privilege
- ☑ Keep systems and software up to date
- ☑ Monitor logs and set up alerts
- ☑ Encrypt data everywhere
- ☑ Regularly assess and test security
- ☑ Have an incident response plan

11. Identity + Access Management (IAM)

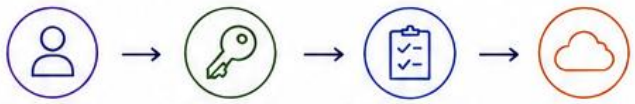
@codedsoul_05

What is IAM?

Identity and Access Management (IAM) is a framework of policies and technologies that ensures the right users have the right access to the right resources at the right time.

Goal: Allow secure access, enforce least privilege, and maintain auditability.

How IAM Works



1. User/Entity requests access
2. IAM authenticates the identity
3. IAM authorizes based on policies
4. Access to resources is granted (or denied)

Always follow the principle of Least Privilege.

Core Concepts

Identity	A user, group, or entity (human or application) that can be authenticated.
Authentication	Verifies who the identity is. (Password, MFA, SSO, Tokens, etc.)
Authorization	Determines what an identity is allowed to do (permissions).
Policies	Rules that define permissions for identities.
Principle of Least Privilege	Grant only the minimum access required to perform a task.

IAM Components

Component	Description
Users	Individual identities that represent a person or application.
Groups	Collection of users. Permissions can be assigned to groups for easier management.
Roles	IAM identities with specific permissions that can be assumed by users or services.
Policies	JSON documents that define permissions. Can be attached to users, groups, or roles.
Permissions	Allow or deny actions on resources. (e.g., read, write, delete)
Identity Providers (IdP)	External systems that authenticate users (SSO, OAuth, SAML, OIDC).
MFA	Adds an extra layer of security during authentication.

Types of IAM Policies

Policy Type	Description
Identity-based Policies	Attached to users, groups, or roles. Define what actions are allowed.
Resource-based Policies	Attached to a resource. Define who can access the resource and what actions are allowed.
Permissions Boundaries	Set the maximum permissions an identity can have (does not grant permissions, only limits).
Service Control Policies (SCPs)	Applied at the organization level. Define the maximum permissions for all accounts (AWS Organizations).

IAM Policy Structure (Example)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket ", "s3:GetObject"],
      "Resource": ["arn:aws:s3:::my-bucket", "arn:aws:s3:::my-bucket/*"]
    }
  ]
}
```

- Policy version
- Statement block
- Effect (Allow/Deny)
- Actions allowed
- Resources the actions apply to

Authentication Methods

- **Passwords** — Traditional username / password login.
- **Multi-Factor Authentication (MFA)** — Adds a second factor (OTP, hardware token, etc.).
- **Single Sign-On (SSO)** — One login to access multiple applications.
- **Federation** — Use external identity providers (Google, Azure AD, etc.).
- **Access Keys / Tokens** — Programmatic access for applications and services.

Best Practices

- [x] Use least privilege access.
- [x] Use groups to manage permissions, not individual users.
- [x] Enable MFA for all users, especially with elevated access.
- [x] Regularly review and audit permissions.
- [x] Use roles for applications and AWS services.
- [x] Rotate access keys and secrets regularly.
- [x] Monitor and log all activities.

Common IAM Services (Examples)

AWS IAM	AWS IAM Identity Center (SSO)	Amazon Cognito	AWS Directory Service
Manage users, groups, roles and policies.	Manage access across multiple AWS accounts and applications.	User sign-up, sign-in, and access management for applications.	Use Managed Microsoft AD or Simple AD in the cloud.

Use Cases

- Grant a developer access to an S3 bucket.
- Allow an EC2 instance to read from DynamoDB.
- Restrict access to production resources.
- Enable SSO for employees across multiple accounts.

Key Takeaway

IAM is the gatekeeper of your cloud environment. Strong identity management and least-privilege access are critical to secure, scalable, and compliant cloud architectures.

12. Containers + Docker

@codedsoul_05

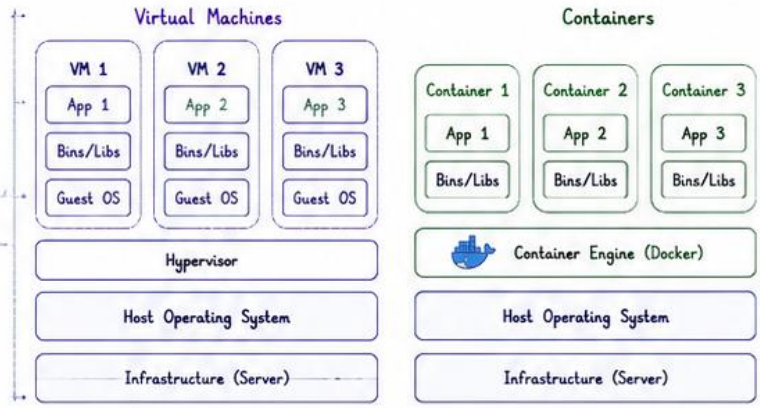
What are Containers?

Containers are lightweight, standalone, executable packages that include everything needed to run a piece of software: code, runtime, system tools, libraries and settings. They ensure consistency across environments.

Why Containers?

- ☑ Works on my machine!
- ☑ Consistency across environments
- ☑ Lightweight & fast
- ☑ Easy to scale and deploy

Containers vs Virtual Machines



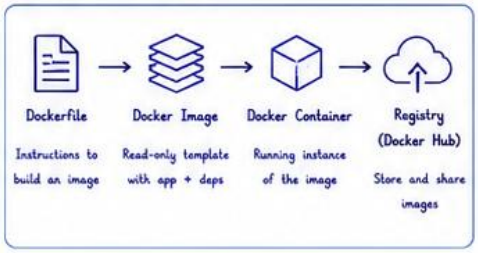
Key Takeaway

Containers share the host OS kernel via the container engine, making them much lighter and faster than VMs.

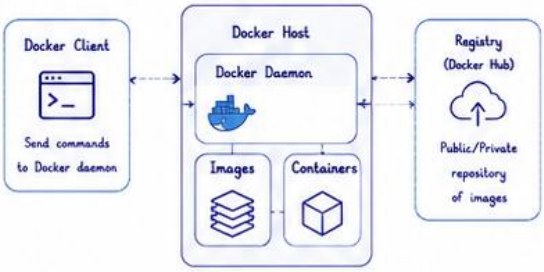
Benefits of Containers

- **Portability** — Build once, run anywhere.
- **Efficiency** — Lightweight, use fewer resources.
- **Scalability** — Easy to scale up and down.
- **Speed** — Fast startup and deployment.
- **Isolation** — Each container is isolated and secure.

How Docker Works



Docker Architecture



Essential Docker Commands

Command	Description
`docker --version`	Check Docker version
`docker pull <image>`	Pull an image from registry
`docker images`	List all images
`docker run <image>`	Run a container from an image
`docker run -d -p 8080:80 <image>`	Run in detached mode and map port
`docker ps`	List running containers
`docker ps -a`	List all containers
`docker stop <container_id>`	Stop a running container
`docker rm <container_id>`	Remove a container
`docker rmi <image_id>`	Remove an image
`docker logs <container_id>`	View container logs
`docker exec -it <container_id> bash`	Open shell inside container
`docker build -t <name>:<tag> .`	Build image from Dockerfile
`docker push <name>:<tag>`	Push image to registry

Dockerfile Basics

Instruction	Purpose
`FROM <image>`	Base image
`WORKDIR <path>`	Set working directory
`COPY <src> <dest>`	Copy files
`ADD <src> <dest>`	Copy & extract files (URL, tar, etc.)
`RUN <command>`	Run commands
`EXPOSE <port>`	Expose container port
`ENV <key> <value>`	Set environment variable
`CMD ["command"]`	Default command
`ENTRYPOINT ["cmd"]`	Executable entry point

Sample Dockerfile (Node.js)

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Container Lifecycle

- **Create** — Create a container
- **Start** — Start the container
- **Stop** — Stop the container
- **Restart** — Restart the container
- **Remove** — Remove the container

Use Cases

- Microservices Architecture
- CI/CD Pipelines
- Cloud Native Applications
- Dev & Test Environments
- Local Development with Consistency

Best Practices

- ☑ Keep images small (use minimal base images)
- ☑ Use .dockerignore to exclude unnecessary files
- ☑ Run containers as non-root user
- ☑ Scan images for vulnerabilities
- ☑ Tag images properly
- ☑ Use multi-stage builds to reduce image size
- ☑ Don't store secrets in images

Key Takeaway

Containers package your application and its dependencies once and run consistently everywhere. Docker makes it easy to build, ship and run containers at scale.

13. Kubernetes + Container Orchestration

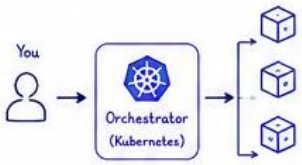
@codedsoul_05

What is Kubernetes?

Kubernetes (K8s) is an open-source container orchestration platform that automates deployment, scaling, management and networking of containerized applications.

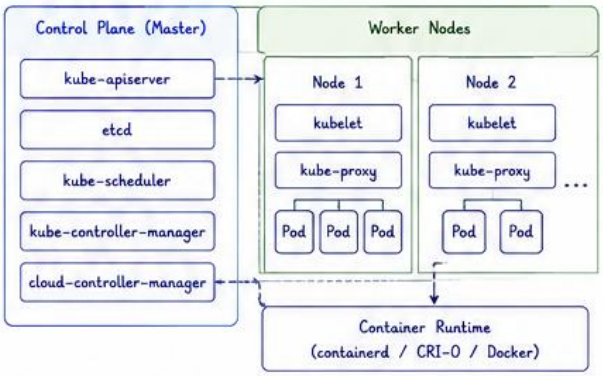
- ☑ Automates operations
- ☑ Self-healing
- ☑ Horizontal scaling
- ☑ High availability
- ☑ Efficient resource utilization

Container Orchestration



You declare the desired state, Kubernetes makes it happen and keeps it that way.

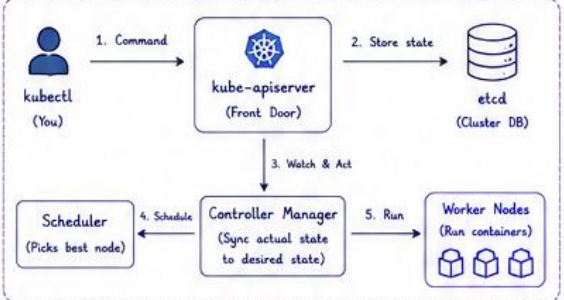
Kubernetes Architecture



Why Kubernetes?

- **Automation** — Automate deployment, scaling and operations.
- **Resilience** — Self-healing and automatic recovery.
- **Portability** — Run anywhere: on-prem, cloud, or hybrid.
- **Efficiency** — Better resource utilization and cost savings.
- **Declarative** — Define desired state, Kubernetes maintains it.

How Kubernetes Works?



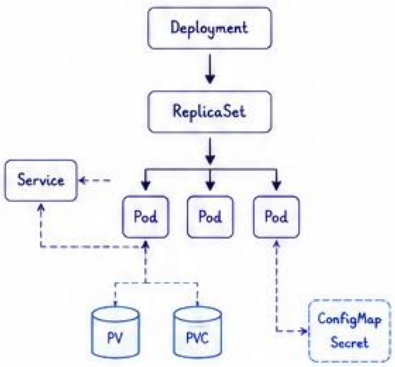
Key Features

- ☑ Automatic bin packing
- ☑ Self-healing (restart, reschedule)
- ☑ Horizontal scaling
- ☑ Rolling updates & rollbacks
- ☑ Service discovery & load balancing
- ☑ Secret & configuration management
- ☑ Storage orchestration
- ☑ Batch execution & CronJobs

Kubernetes Objects (Building Blocks)

Object	Purpose
Pod	Smallest deployable unit. One or more containers.
Deployment	Manages ReplicaSets and provides declarative updates.
ReplicaSet	Ensures specified number of pod replicas are running.
Service	Stable network endpoint to access pods.
Namespace	Virtual cluster inside a physical cluster.
ConfigMap	Store configuration as key-value pairs.
Secret	Store sensitive data like passwords, tokens.
PersistentVolume (PV)	Piece of storage in the cluster.
PersistentVolumeClaim (PVC)	Request storage by users.
Ingress	HTTP/HTTPS routing to services.

Kubernetes Object Relationship



Common kubectl Commands

Command	Description
kubectl get all	List all resources
kubectl get pods	List pods
kubectl describe <pod>	Show details of a pod
kubectl logs <pod>	View logs of a pod
kubectl apply -f file.yaml	Create/update from file
kubectl delete -f file.yaml	Delete resources from file
kubectl exec -it <pod> -- sh	Open shell in a pod
Rubectl scale deployment <name> --replicas=3	Scale deployment
kubectl rollout status deployment/<name>	Check rollout status
kubectl get nodes	List all nodes

Kubernetes Resource Hierarchy



Sample Deployment YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  labels:
    app: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
          - containerPort: 80
```

Common Patterns

- **Rolling Update** — Update application without downtime.
- **Blue/Green Deployment** — Run two identical environments and switch traffic.
- **Canary Deployment** — Release to small subset of users first.
- **Auto Scaling** — Scale pods based on CPU / Metrics.

Kubernetes Components Explained

Component	Description
kube-apiserver	API server; validates & processes requests.
etcd	Consistent and highly-available key-value store.
kube-scheduler	Assigns pods to nodes.
kube-controller-manager	Runs controllers (Node, ReplicaSet, Endpoint, etc.)
kubelet	Agent on each node; ensures containers are running.
kube-proxy	Maintains network rules on nodes.
Container Runtime	Runs containers (containerd, CRI-O, Docker).

Best Practices

- ☑ Use namespaces to isolate environments.
- ☑ Always set resource requests & limits.
- ☑ Use liveness & readiness probes.
- ☑ Store config in ConfigMaps & secrets.
- ☑ Use Rolling Updates for zero downtime.
- ☑ Enable RBAC for secure access.
- ☑ Monitor with Prometheus & Grafana.
- ☑ Keep cluster and images up to date.

Use Cases

- Microservices Orchestration
- CI/CD Pipelines
- Multi-tenant Platforms
- Batch & Scheduled Jobs
- Cloud-native Applications

Key Takeaway Kubernetes takes care of the heavy lifting of running containerized applications at scale so you can focus on building your app, not managing infrastructure.

14. Serverless Computing + Functions

@codedsoul_05

What is Serverless?

Serverless is a cloud computing model where the cloud provider manages the infrastructure. You focus only on writing code and business logic. No need to provision, manage or scale servers.

You write code.
We handle the rest!

How Serverless Works



1. Event Trigger (HTTP request, DB change, S3 upload, etc.)
2. Function Executes (Only when needed)
3. Function Ends (No server to manage)
4. Pay for Value (Per request + compute time)

Scale automatically from zero to thousands of instances.

Key Benefits

- [x] No server management
- [x] Automatic scaling
- [x] Pay per use (cost-efficient)
- [x] High availability & fault tolerance
- [x] Faster time to market
- [x] Focus on business logic

Serverless Use Cases

- **Web / Mobile Backends** — APIs, authentication, data processing
- **File Processing** — Image resizing, video transcoding
- **Scheduled Tasks** — Cron jobs, report generation
- **Real-time Data Processing** — Stream processing, IoT data
- **Chatbots & Notifications** — Event-driven messaging

Serverless Services (Examples)

Cloud Provider	Function Service	Event Sources (Examples)
aws	AWS Lambda	API Gateway, S3, DynamoDB, SQS, EventBridge, Cognito, etc.
	Azure Functions	HTTP, Storage, Service Bus, Event Grid, Cosmos DB, etc.
	Cloud Functions (GCP)	Cloud Storage, Pub/Sub, Firestore, HTTP, etc.
	IBM Cloud Functions	Cloud Object Storage, API, Kafka, Dec Kafka, etc.

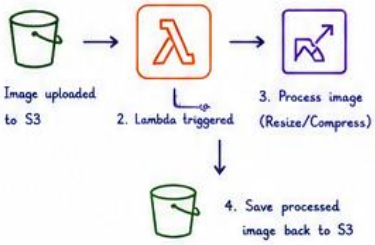
Serverless vs Traditional

Feature	Serverless	Traditional
Infrastructure	Managed by provider	You manage
Scaling	Automatic	Manual
Billing	Pay per request & compute time	Pay for provisioned resources
Idle Cost	No	Yes
Maintenance	Minimal	High
Speed to Market	Faster	Slower
Use Case Fit	Event-driven, short tasks	Long-running, stateful apps

Key Concepts

Function	A piece of code that runs in response to an event.
Event Source	The trigger that invokes the function.
Handler	The entry point in your code.
Execution Context	Environment in which the function runs (memory, timeout, env vars).
Stateless	Functions should not rely on local state. Use external storage.
Cold Start	Delay when a function is invoked after being idle.

Example Flow (Image Resize)



1. Image uploaded to S3
2. Lambda triggered
3. Process image (Resize/Compress)
4. Save processed image back to S3

All happens automatically.
No server to manage!

Sample Serverless Function (Python - AWS Lambda)

```python
import json

def lambda_handler(event, context):
    name = event.get('name', 'World')
    message = f'Hello, {name}!'
    return {
        'statusCode': 200,
        'body': json.dumps({
            'message': message
        })
    }
```

← Input event (Trigger data)

Your code runs here

← Return response

Event Sources (Common)

API Gateway (HTTP) · S3 (Upload) · DynamoDB (Streams) · SQS / SNS (Messages) · EventBridge (Scheduled) · ... and more

Serverless Best Practices

- [x] Keep functions small and focused.
- [x] Use least privilege IAM roles.
- [x] Set appropriate timeout and memory.
- [x] Use environment variables for configuration.
- [x] Handle errors and retries gracefully.
- [x] Monitor with logs, metrics and alerts.
- [x] Optimize for cold start (choose right runtime, package size).

Serverless Pricing (Typical Model)

Cost = Requests (Number of invocations) × Duration (Execution time in seconds/100ms) × Memory (Allocated memory) + Additional Charges (Data transfer, Storage, etc.)

You pay only when your code runs!

Things to Know / Limitations

- ✗ Executions time limits (e.g., 15 min in AWS Lambda)
- ✗ Not ideal for long-running, stateful applications
- ✗ Cold start can impact latency
- ✗ Vendor lock-in (use abstractions if possible)

Key Takeaway — Serverless lets you build and run applications without managing servers. It's event-driven, scalable, and cost-efficient — perfect for modern cloud applications.

15. Cloud Monitoring + Logging

@codedsoul_05

What is Monitoring?

Monitoring is continuously collecting, analyzing and visualizing metrics to understand the health, performance and availability of cloud resources and applications.

Goal: Detect issues early, understand system behavior, and ensure reliability.

What is Logging?

Logging is collecting, storing and analyzing log data generated by applications, systems and services to troubleshoot issues, audit activity and gain insights.

Goal: Record what happened for debugging, audit and compliance.

Monitoring + Logging Architecture



Collect → Store → Analyze → Act → Improve

Why it Matters?

- [x] High availability and reliability
- [x] Faster issue detection
- [x] Better performance optimization
- [x] Capacity planning
- [x] Security & compliance
- [x] Business insights

Types of Monitoring

Type	Description
Infrastructure Monitoring	Monitor VMs, servers, storage, network, CPU, memory, disk, etc.
Application Monitoring (APM)	Monitor application performance, transactions, response time, errors.
Synthetic Monitoring	Simulate user actions to monitor availability and performance.
Real User Monitoring (RUM)	Monitor real user experience in web & mobile apps.
Log Monitoring	Collect and analyze logs to detect issues and track events.
Security Monitoring	Monitor threats, anomalies, access logs, and audit activities.

Key Metrics to Monitor

Category	Examples
Compute	CPU Utilization, Load Average, Memory Usage, Disk I/O
Storage	Disk Space, IOPS, Latency, Throughput
Network	Bandwidth, Latency, Packet Loss, Connections
Application	Response Time, Error Rate, Throughput, Apdex Score
Database	Query Latency, Connections, Cache Hit Ratio, Replication Lag
Business	Sign-ups, Orders, Revenue, Conversion Rate

Cloud Monitoring & Logging Tools (Examples)

Cloud Provider	Monitoring	Logging
aws	Amazon CloudWatch	CloudWatch Logs
Azure	Azure Monitor	Log Analytics
Google Cloud	Cloud Monitoring	Cloud Logging
	Datadog, New Relic, Dynatrace	ELK Stack, Splunk, Grafana Loki

Logs: Structure & Levels

Log Structure (Example)

```
{
  "timestamp": "2025-05-24T12:30:45Z",
  "level": "ERROR",
  "service": "payment-service",
  "message": "Payment failed",
  "trace_id": "abc123...",
  "user_id": "u_789",
  "metadata": {
    "amount": 2500,
    "currency": "INR"
  }
}
```

Log Levels (Low → High)

DEBUG	Detailed information
INFO	General information
WARN	Warning messages
ERROR	Error events
FATAL	Critical failures

Use consistent log format (JSON) and include correlation/trace IDs.

Alerts: Best Practices

- Alert on symptoms, not on root causes.
- Avoid alert fatigue. Set meaningful thresholds.
- Use severity levels (P1, P2, P3, P4).
- Include runbook links in alerts.
- Route alerts to the right person at the right time.
- Review and tune alerts regularly.

Alert Example

Name: High CPU Usage
Severity: P2 (Warning)
Condition: CPU > 80% for 5 minutes
Resource: i-0abcd1234
action: Notify #ops
Channel: Slack + Email

Dashboards: What to Include?

Service Health | Performance (Metrics) | Errors & Exceptions | Traffic & Usage | Infrastructure Health | Business Metrics

Keep dashboards simple, actionable and role-based.

Logging Best Practices

- [x] Log what is important, not everything.
- [x] Use structured logs (JSON) with consistent schema.
- [x] Include context: trace_id, user_id, request_id.
- [x] Do not log sensitive data (PII, passwords, tokens).
- [x] Centralize logs and set retention policies.
- [x] Index logs for fast search and analysis.
- [x] Monitor log ingestion and storage usage.

Log Retention (Example)

Hot Storage	7 days
Warm Storage	30 days
Cold Storage	90 days
Archive	1 year

Common Monitoring Use Cases

Detect service outages | Identify performance bottlenecks | Troubleshoot errors | Capacity planning | Security & threat detection

Integrations

Incident Management (PagerDuty, Opsgenie) | Communication (Slack, Teams) | ITSM (ServiceNow) | Auto Scaling (AWS ASG, K8s HPA)

Key Takeaway Monitoring tells you about the future, logging tells you about the past. Together they give you full observability and help you build reliable, scalable and secure cloud applications.

16. DevOps + CI/CD in Cloud

@codedsoul_05

What is DevOps?

DevOps is a culture and set of practices that combines Development (Dev) and Operations (Ops) to deliver software faster, more reliably and with high quality.

Goal: Deliver value to users continuously with automation, collaboration and feedback.

DevOps Principles

Collaboration	Automation	Fast Delivery	Visibility	Reliability	Feedback
Break silos and work together	Automate repetitive processes	Deliver small changes frequently	Make processes and results visible	Build quality in and ensure stability	Learn from feedback and improve

Why DevOps in Cloud?

- [x] Scalability & elasticity
- [x] Pay-as-you-go infrastructure
- [x] High availability & reliability
- [x] Global reach
- [x] Automation at every layer
- [x] Faster time to market
- [x] Better collaboration

CI/CD Explained

CI (Continuous Integration): Developers integrate code frequently into a shared repository. Automated builds and tests verify the changes.

CD (Continuous Delivery/Deployment): Code is automatically prepared, tested and deployed to environments. Delivery is automated; deployment can be automated or manual.

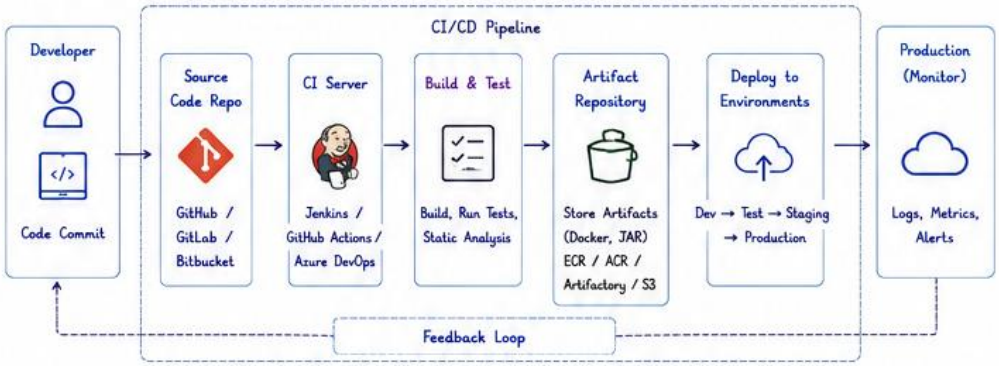
CI/CD Pipeline – Overview

Code	Build	Test	Package	Deploy	Monitor
Write code & commit	Compile code, resolve deps	Run unit, integration, security tests	Create artifact (Docker image, JAR, etc.)	Deploy to staging / production	Monitor, log and get feedback

Benefits of DevOps + CI/CD

- [x] Faster releases and feature delivery
- [x] Improved code quality
- [x] Early bug detection
- [x] Consistent and repeatable deployments
- [x] Reduced manual errors
- [x] Better collaboration and productivity
- [x] Lower costs

Typical CI/CD Flow in Cloud



Popular CI/CD Tools in Cloud

Tool	Cloud / Provider	Highlights
Jenkins	AWS, Azure, GCP (On EC2 / VM)	Open source, highly customizable, large ecosystem
GitHub Actions	GitHub (Cloud)	Easy integration with GitHub, great for modern workflows
GitLab CI/CD	GitLab (SaaS / Self-hosted)	Built-in CI/CD, integrated with GitLab ecosystem
AWS CodePipeline	AWS	Fully managed CI/CD service with AWS integrations
Azure DevOps Pipelines	Microsoft Azure	CI/CD, Boards, Repos, Artifacts in one platform
Google Cloud Build	Google Cloud	Fast, secure builds using Kubernetes

Infrastructure as Code (IaC)

Manage and provision cloud resources through code for consistency and repeatability.

- Terraform (Multi-cloud)
- AWS CloudFormation
- Azure Bicep / ARM Templates
- Google Deployment Manager

IaC + CI/CD = Fully Automated Infrastructure & Deployments

Best Practices

- [x] Keep builds small and fast
- [x] Automate tests at all stages
- [x] Use version control for everything
- [x] Store artifacts in centralized repository
- [x] Deploy to immutable infrastructure
- [x] Use feature flags for safe releases
- [x] Implement proper rollback strategy
- [x] Monitor everything in production
- [x] Follow security best practices (SAST, Secrets management, least privilege)

Environments in a Typical Flow

Development	Testing	Staging	Production
For developers to build & test	QA testing & automation	Pre-production validation	Live environment for users

Monitoring & Logging in DevOps

CloudWatch	Azure Monitor	Google Cloud Monitoring	ELK Stack	Datadog
Logs, Metrics, Alarms	Metrics, Logs, Alerts	Metrics, Logs, Dashboards	Elasticsearch, Logstash, Kibana	Monitoring, Logs, APM

Example: CI/CD on AWS (High Level)



DevOps Culture

People	Collaboration & ownership
Process	Continuous improvement
Technology	Automation & measurement

Key Takeaway DevOps + CI/CD in Cloud enables continuous delivery of high-quality software with automation, scalability, reliability and security at every step.

17. Cloud Cost Management + Optimization

@codedsoul_05

What is Cloud Cost Management?

Cloud Cost Management is the practice of monitoring, controlling and optimizing cloud spend while maximizing business value and performance.

Goal: Get the most value from every dollar spent in the cloud.

Cloud Cost Management Lifecycle



1. **Visibility** — Understand where and how money is being spent
2. **Measurement** — Measure usage, costs and allocate to teams/projects
3. **Analysis** — Identify cost drivers, trends, anomalies & waste
4. **Optimization** — Implement right-sizing, automation and discounts
5. **Governance** — Set policies, budgets and continuously improve

Why it Matters?

- ☑ Reduce unneccesary spend
- ☑ Improve resource utilization
- ☑ Free up budget for innovation
- ☑ Increase financial accountability
- ☑ Better forecasting & planning
- ☑ Sustainable & scalable growth

Key Cost Components

- **Compute** — VMs, Containers, Serverless, Databases
- **Storage** — Block, Object, Archive, Snapshots
- **Data Transfer** — Inbound / Outbound / Inter-Region
- **Networking** — Load Balancers, NAT, VPN, CDN, IPs
- **Managed Services** — Databases, AI/ML, Monitoring, etc.
- **Licenses & Others** — Marketplace, 3rd party tools, Support

Cost Visibility Tools (Examples)

Cloud Provider	Native Tools	Highlights
aws	AWS Cost Explorer AWS Budgets AWS Cost & Usage Report	Visualize spend, set budgets, alerts, forecasts, detailed reports
Azure	Azure Cost Management + Billing Azure Budgets Azure Advisor	Cost analysis, budgets, recommendations, chargeback
Google Cloud	Google Cloud Billing Cost Table Recommender	Cost breakdown, filtering, commitment tracking, insights
	CloudHealth Flexera One Spot by NetApp	Multi-cloud visibility, showback/ chargeback, optimization

Common Causes of High Cloud Costs

- ✗ Running oversized resources
- ✗ Idle or unused resources
- ✗ Unattached storage & snapshots
- ✗ Over-provisioned environments
- ✗ Lack of monitoring & budgets
- ✗ Data transfer between regions
- ✗ Not using discounts & reserved options
- ✗ Orphaned resources

Cost Optimization Strategies

Right-sizing	Use appropriate instance types & sizes.
Scheduling	Start/stop resources when not in use.
Auto Scaling	Scale out/in based on demand.
Use Managed Services	Offload undifferentiated heavy lifting.
Reserved / Committed Use	Use Reserved Instances, Savings Plans (AWS) or Reserved VM Instances (Azure).
Spot / Preemptible Instances	Use for fault-tolerant, non-critical workloads.
Storage Optimization	Choose right storage class & lifecycle policies.
Eliminate Waste	Remove idle resources, orphaned IPs, volumes, snapshots.
Use Discounts	Leverage enterprise agreements, credits, offers.

Cost Optimization Framework (5R)

- **R Report** → Get visibility. What are we spending on? Use dashboards, alerts, tagging.
- **R Reclaim** → Find & remove waste. Idle, unattached, underutilized resources.
- **R Right-size** → Match resources with actual needs. Use the right size, not the biggest.
- **R Reserve** → Commit for steady workloads. Use Reserved Instances / Savings Plans.
- **R Rearchitect** → Improve architecture to be more efficient & cost effective.

Tagging Best Practices

- ☑ Use consistent tagging across all resources
- ☑ Include: Environment, Owner, Project, CostCenter, App, Purpose
- ☑ Enables cost allocation & chargeback
- ☑ Helps in automation & policy enforcement

Example Tags

Key	Value (Example)
Environment	Production
Owner	Team-A
Project	Payment-Service
CostCenter	CC-1234
Application	Web-App
Purpose	Customer Facing

Cost Allocation & Chargeback



- **Tag Resources** — Tag all resources with meaningful keys
- **Allocate Costs** — Assign costs to teams/ projects using tags
- **Chargeback / Showback** — Charge or show costs to business units

Budgeting & Alerts



1. Set Budget
2. Monitor Usage
3. Alert on Threshold

Example: Alert when cost > $500

AWS Budgets | Azure Budgets | GCP Budget Alerts

Cost Anomaly Detection



Detect unusual spikes or unexpected cost increases and take action early.

Tools: AWS Cost Anomaly Detection
Azure Cost Anomaly
GCP Anomaly Detection

Commitement Options Comparison

Option	Best For	Commitment	Discount	Flexibility
On-Demand	Short term, unpredictable	No	No	High
Reserved Instances (AWS)	Steady, 1-3 yrs workloads	1 or 3 years	Up to 72%	Low to Medium
Savings Plans (AWS)	Compute usage flexibility	1 or 3 years	Up to 66%	High
Reserved VM Instances (Azure)	Steady workloads	1 or 3 years	Up to 72%	Low
Committed Use Discounts (GCP)	Sustained use	1 or 3 years	Up to 57%	High

Optimization Tools & Services

Provider	Services	Purpose
aws	AWS Trusted Advisor Compute Optimizer AWS Well-Architected Tool	Best practices, rightsizing recommendations, cost optimization guidance
Azure	Azure Advisor Azure Monitor Recommendations	Cost recommendations, best practices, reliability
Google Cloud	Recommender Cost Optimization Scorecard	Rightsizing, committed use recommendations, insights

Cost Management Best Practices

- ☑ Turn on cost visibility on day 1
- ☑ Tag everything consistently
- ☑ Set budgets and alerts
- ☑ Review costs regularly
- ☑ Rightsize resources continuously
- ☑ Automate scheduling & scaling
- ☑ Use reserved/commitment for steady workloads
- ☑ Delete what you don't use
- ☑ Use lifecycle policies for storage
- ☑ Monitor & optimize data transfer
- ☑ Follow FinOps culture & collaboration



FinOps Principles

- Teams collaborate
- Everyone takes ownership
- Decisions are driven by business value
- Report & improve continuously

Key Takeaway — Great engineering in the cloud is not just about building — it's about building it cost-effectively.
Visibility → Optimization → Governance = Maximum value from every cloud dollar.

18. High Availability + Disaster Recovery

@codedsoul_05

What is High Availability (HA)?

High Availability ensures that applications remain available and operational for as long as possible, minimizing downtime due to failures.

Goal: Maximize uptime and eliminate single points of failure.

HA vs DR

Aspect	High Availability (HA)	Disaster Recovery (DR)
Purpose	Keep systems running during component failures	Recover systems after a major disaster
Focus	Minimize downtime	Restore operations
Scope	Within a region	Across regions
RPO	Near zero	Minutes to hours
RTO	Seconds to minutes	Minutes to hours
Examples	Load balancers, Auto Scaling, Multi-AZ, Failover	Backups, Cross-region replication, Pilot light, Warm standby

Why It Matters

- ☑ Ensures business continuity
- ☑ Protects revenue and reputation
- ☑ Meets SLAs and compliance
- ☑ Mitigates risk of data loss
- ☑ Increases customer trust

High Availability Strategies

Eliminate Single Points of Failure
Design systems so no single component failure can bring down the application.

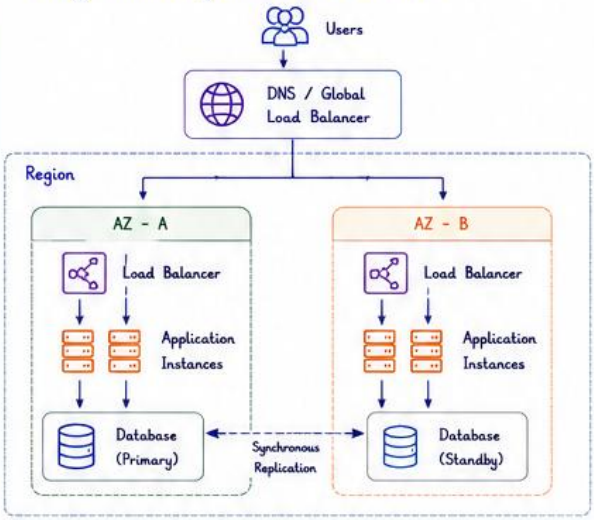
Redundant Infrastructure
Use multiple instances, AZs and resources.

Health Checks & Auto Recovery
Continuously monitor and automatically replace failed components.

Load Balancing
Distribute traffic across healthy instances.

Scaling
Scale out/in to handle demand and failures.

High Availability Architecture (Multi-AZ)



Disaster Recovery Overview

1. **Detect Disaster**
   Identify outage in primary region.
2. **Failover**
   Route traffic to secondary region.
3. **Recover & Continue**
   Restore services and continue operations.
4. **Failback (Optional)**
   Return to primary region when it is operational.

HA Best Practices

- ☑ Deploy across multiple AZs.
- ☑ Use managed services with built-in HA.
- ☑ Enable health checks and automatic failover.
- ☑ Regularly test failover scenarios.
- ☑ Monitor constantly and set alerts.
- ☑ Keep dependencies (DB, Cache, Queue) highly available.

Disaster Recovery Strategies

Strategy	Description	RPO	RTO	Cost
Backup & Restore	Restore from backups in another region.	Hours - Days	Hours - Days	Low
Pilot Light	Core infrastructure in DR region (always on). Scale up during DR.	Minutes - Hours	Minutes - Hours	Medium
Warm Standby	Scaled down copies of production in DR region.	Minutes	Minutes	High
Multi-Site / Active-Active	Actively running in multiple regions. Traffic routed intelligently.	Near Zero	Seconds	Very High

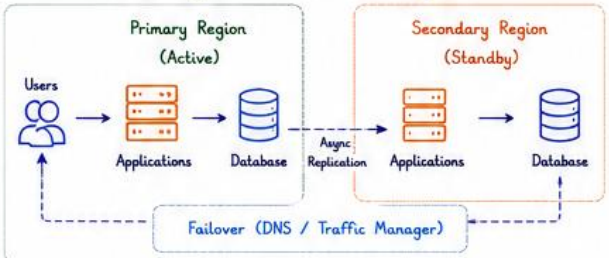
Key DR Metrics

RPO (Recovery Point Objective)
Maximum acceptable amount of data loss measured in time.

RTO (Recovery Time Objective)
Maximum acceptable downtime before services are restored.

Lower RPO and RTO = Better resilience (higher cost).

Disaster Recovery Architecture (Cross-Region)



Data Protection Techniques

Automated Backups
Regular backups to durable storage.

Cross-Region Replication
Replicate data to another region.

Point-in-Time Recovery
Restore to any point from backups.

Snapshots
Capture point-in-time copies of data.

Testing & Validation

- ☑ Perform regular DR drills.
- ☑ Validate backups and restores.
- ☑ Test failover and failback processes.
- ☑ Document runbooks and procedures.
- ☑ Review and update plans regularly.

Common Cloud Services for HA & DR (Examples)

Cloud Provider	HA Services	DR Services
aws	Multi-AZ, Auto Scaling, ELB, RDS Multi-AZ, EFS, ElastiCache	S3 Cross-Region Replication, AWS Backup, Route 53, Elastic DR (CloudEndure)
Azure	Availability Zones, VM Scale Sets, Azure Load Balancer, SQL DB Geo-Replication	Azure Site Recovery, Geo-Redundant Storage, Traffic Manager
Google Cloud	Regional MIG, Cloud Load Balancing, Cloud SQL HA, Spanner	Cross-Region Replication, Backup & DR, Cloud DNS, Cloud Armor

HA/DR Design Checklist

- ☑ Identify critical applications and data.
- ☑ Set RPO and RTO targets.
- ☑ Choose the right HA and DR strategy.
- ☑ Design for multi-AZ and cross-region.
- ☑ Automate backups and replication.
- ☑ Implement monitoring and alerting.
- ☑ Document, test, and improve continuously.

Common Mistakes to Avoid

- ✗ Relying on a single AZ or region.
- ✗ No backups or untested restores.
- ✗ Ignoring dependencies (DNS, DB, Cache).
- ✗ Manual failover with no automation.
- ✗ Not testing DR plans.

Key Takeaway

High Availability keeps your application running during failures.
Disaster Recovery gets you back in business after a disaster.
Plan, automate, test, and monitor to build resilient and reliable systems in the cloud.

19. Real-World Cloud Architecture Problems

@codedsoul_05

Solving real-world problems is the best way to master cloud architecture.
Understand requirements → Break down → Design → Optimize → Secure → Monitor

Key Takeaway

There is no single correct architecture. Choose the right services based on requirements, trade-offs & cost.

• How to Approach Any Problem

- ☑ Understand requirements & constraints
- ☑ Identify functional & non-functional needs
- ☑ Estimate scale (users, data, RPS, growth)
- ☑ Design high-level architecture
- ☑ Consider Security, HA, Scalability, Cost
- ☑ Review, optimize & monitor

• Common Non-Functional Requirements

High Availability | Scalability | Performance | Security | Durability | Cost Optimization

1. Global E-Commerce Website

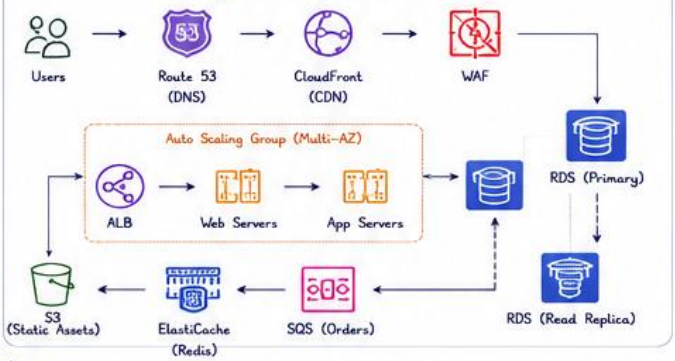
Problem

Build an e-commerce platform that can handle millions of users, high traffic during sales, and provide a smooth shopping experience globally.

Key Requirements

- High availability
- Auto scalability
- Low latency (global)
- Secure payments
- Durable data

High Level Architecture (AWS)



Why This Design?

- ☑ CloudFront for global performance
- ☑ ALB + Auto Scaling for availability & scale
- ☑ RDS Multi-AZ for managed HA
- ☑ S3 for durable & cost-effective storage
- ☑ ElastiCache for session & caching
- ☑ SQS to decouple order processing

Other Options

Use DynamoDB for orders, Aurora Serverless v2 for variable workloads.

2. Real-Time Chat Application

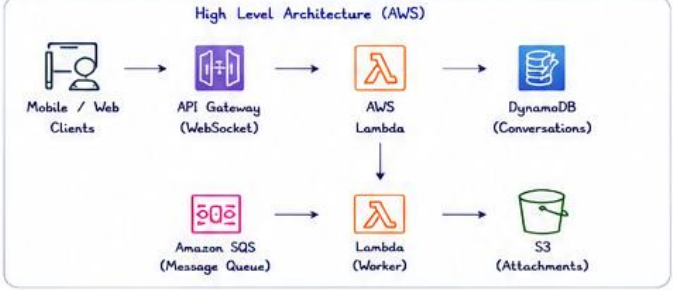
Problem

Build an real-time chat app with 1M+ daily active users where messages are delivered instantly.

Key Requirements

- Real-time messaging
- High availability
- Horizontal scalability
- Store chat history
- Low latency

High Level Architecture (AWS)



Why This Design?

- ☑ WebSocket via API Gateway for real-time
- ☑ Lambda for auto-scaling & stateless compute
- ☑ DynamoDB for low-latency & massive scale
- ☑ SQS for reliable async processing
- ☑ S3 for storing media files

Other Options

Use ElastiCache (Redis) for presence, Amazon MQ for managed message broker.

3. Data Analytics Pipeline

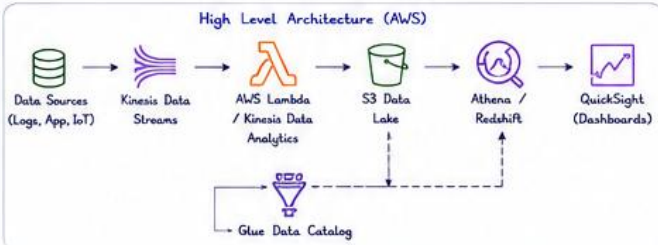
Problem

Ingest data from multiple sources, process it and generate insights with dashboards.

Key Requirements

- Handle large volume data
- Scalable data processing
- Secure & reliable storage
- Near real-time analytics

High Level Architecture (AWS)



Why This Design?

- ☑ Kinesis for real-time ingestion
- ☑ S3 as central data lake (durable & cheap)
- ☑ Glue Catalog for data discovery
- ☑ Athena/Redshift for analysis
- ☑ QuickSight for visualization

Other Options

Use MSK (Kafka) for streaming, EMR for heavy ETL workloads.

4. Media Streaming Platform

Problem

Build a video streaming platform that can handle video uploads, transcoding, and smooth playback to users worldwide.

Key Requirements

- High availability
- Global delivery
- Transcoding videos
- Secure content



Why This Design?

- ☑ S3 for durable, scalable video storage
- ☑ MediaConvert for high-quality transcoding
- ☑ MediaPackage for secure packaging
- ☑ CloudFront for global low-latency delivery
- ☑ Encryption for content protection

Other Options

Use Elastic Transcoder (legacy), DRM for premium content.

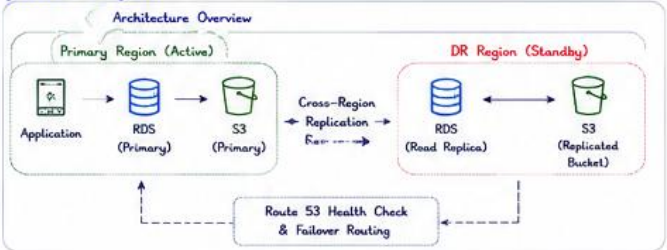
5. Backup & Disaster Recovery (Cross-Region)

Problem

Design a DR solution so that critical data & applications can be recovered in case of a regional failure.

Key Requirements

- ☑ High RPO & RTO
- ☑ Cross-region replication
- ☑ Data durability
- ☑ Automation



Why This Design?

- ☑ RDS Read Replica for DR with minimal RPO
- ☑ S3 Cross-Region Replication for data safety
- ☑ Route 53 Failover for quick traffic switch
- ☑ Regular backups & automation for recovery

Other Options

Use Aurora Global Database, Backup to Glacier for long-term retention.

General Best Practices

- ☑ Design for failure, not just success.
- ☑ Use managed services to reduce ops.
- ☑ Enable monitoring, logging & alerting.
- ☑ Test DR plans regularly.
- ☑ Optimize cost without compromising reliability.

Common Trade-offs

- Consistency vs Availability
- Performance vs Cost
- Complexity vs Flexibility

Useful Services (Across Clouds)

AWS | Azure | GCP

20. TOP 100 CLOUD COMPUTING INTERVIEW QUESTIONS

+ QUICK REVISION CHEAT SHEET

Prepare Smart.
Revise Fast.
Crack Confidently!

TOP 100 CLOUD INTERVIEW QUESTIONS (Categorized)

01 Cloud Fundamentals

1. What is Cloud Computing?
2. What are the characteristics of Cloud?
3. What are the service models?
4. What are the deployment models?
5. What is the difference between CAPEX and OPEX?
6. What is multitenancy?
7. What is elasticity?
8. What is Idempotency?
9. What is a Region, AZ and Edge Location?
10. What is Shared Responsibility Model?

02 Compute Services

11. What is the difference between EC2 and ECS?
12. What are EC2 instance types?
13. What is Auto Scaling?
14. What is Lambda / Serverless?
15. What is Containerization?
16. What is the difference between Docker and Virtual Machines?
17. What is a Load Balancer?
18. What is an AMI?
19. What is Spot Instance?
20. What is a Placement Group?

03 Storage Services

21. What is object storage?
22. What is the difference between S3 and EBS?
23. What is EFS?
24. What is Glacier?
25. What is the storage classes in S3?
26. What is lifecycle policy?
27. What is versioning?
28. What is a Snapshot?
29. What is block storage?
30. How does data durability work in the cloud?

04 Networking

31. What is VPC?
32. What are Subnets?
33. What is the difference between Public and Private Subnet?
34. What is NAT Gateway?
35. What is Internet Gateway?
36. What is VPC Peering?
37. What is Transit Gateway?
38. What is Route 53?
39. What is a Security Group?
40. What is NACL?

05 Databases

41. What is RDS?
42. What is the difference between SQL and NoSQL?
43. What is DynamoDB?
44. What is Read Replica?
45. What is Sharding?
46. What is Indexing?
47. What is Data Consistency?
48. What is Connection Pooling?
49. What is Aurora?
50. What is Multi-AZ DB?

06 Security

51. What is IAM?
52. What is the difference between User, Role and Policy?
53. What is MFA?
54. What is least privilege?
55. What is KMS?
56. What is Security Group?
57. What is WAF?
58. What is Shield?
59. What is Secrets Manager?
60. What is Compliance in Cloud?

07 DevOps & CI/CD

61. What is DevOps?
62. What is CI/CD?
63. What is the difference between CodePipeline and CodeBuild?
64. What is Jenkins?
65. What is Infrastructure as Code?
66. What is Docker?
67. What is Kubernetes?
68. What are Blue/Green Deployments?
69. What is Canary Deployment?
70. What is Rollback?

08 Monitoring & Logging

71. What is CloudWatch?
72. What is the use of CloudWatch Logs?
73. What is CloudTrail?
74. What is a Metric?
75. What is an Alarm?
76. What is X-Ray?
77. What is a Dashboard?
78. What is centralized logging?
79. What is observability?
80. What is synthetic monitoring?

09 High Availability & DR

81. What is High Availability?
82. What is Multi-AZ?
83. What is Cross-Region Replication?
84. What is RPO?
85. What is RTO?
86. What is Backup?
87. What is Disaster Recovery?
88. What is Failover?
89. What is Active-Passive architecture?
90. What is Active-Active architecture?

10 Cost Management

91. What is cost optimization?
92. What is Right-sizing?
93. What are Reserved Instances?
94. What is Spot Instance?
95. What is Auto Scaling for cost optimization?
96. What is Cost Explorer?
97. What is Tagging and why is it important?
98. What is Cloud Budget?
99. What is Showback vs Chargeback?
100. How do you identify idle resources?

QUICK REVISION CHEAT SHEET

1. Service Models

- **IaaS** – Infrastructure as a Service (EC2, VMs, Storage)
- **PaaS** – Platform as a Service (App Engine, Beanstalk)
- **SaaS** – Software as a Service (Google Workspace, Office 365)

2. Deployment Models

- **Public Cloud** – Owned by cloud providers
- **Private Cloud** – Dedicated to one organization
- **Hybrid Cloud** – Combination of both
- **Multi Cloud** – Using multiple cloud providers

3. Key Cloud Benefits

- ☑ Pay-as-you-go
- ☑ Scalability & Elasticity
- ☑ High Availability & Reliability
- ☑ Global Reach
- ☑ Reduced CAPEX
- ☑ Faster Time to Market

4. Shared Responsibility

Responsibility	On-Prem	Cloud
Data	You	You
Applications	You	You
OS / Middleware	You	You
Runtime	You	Provider
Virtualization	You	Provider
Servers	You	Provider
Storage	You	Provider
Networking	You	Provider
Data Centers	You	Provider

5. Common Services

- Compute : EC2, Lambda, ECS
- Storage : S3, EBS, EFS, Glacier
- Database : RDS, DynamoDB, Aurora
- Networking : VPC, Route 53, ELB
- Security : IAM, KMS, WAF, Shield
- Monitoring : CloudWatch, CloudTrail
- DevOps : CodePipeline, CloudFormation
- Analytics : Redshift, Athena, EMR

Architecture Best Practices

- ☑ Design for failure
- ☑ Use Auto Scaling & Load Balancing
- ☑ Decouple components (Microservices)
- ☑ Use Managed Services
- ☑ Implement Caching
- ☑ Follow Security Best Practices
- ☑ Monitor & Alert
- ☑ Optimize for Cost
- ☑ Backup & Test DR regularly

Security Best Practices

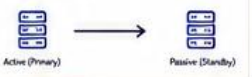
- ☑ Enable MFA
- ☑ Use Least Privilege Access
- ☑ Encrypt data in transit & at rest
- ☑ Rotate keys & secrets
- ☑ Use Security Groups & NACLs
- ☑ Enable logging & monitoring
- ☑ Regular vulnerability scanning
- ☑ Keep systems updated

High Availability Patterns

Multi-AZ



Active-Passive



Active-Active



DR Quick Reference

Metric	Meaning
RPO	Maximum acceptable data loss
RTO	Maximum acceptable downtime
Backup	Copy of data for recovery
Failover	Switch to standby environment

Cost Optimization Tips

- Right-size resources
- Delete unused resources
- Use Auto Scaling
- Use Reserved/Spot Instances
- Use Lifecycle policies for storage
- Use Cloud Budget & Alerts
- Tag resources
- Review & Optimize regularly

Pro Tip

Don't just memorize answers. Understand concepts, use real-world examples, and keep practicing!

Commonly Asked to Discuss

- A real-world architecture you designed
- A challenging problem you solved in cloud
- How you optimized cost / performance
- Your approach to security in cloud

Final Checklist Before Interview

- ✔ Revise core concepts
- ✔ Practice architecture diagrams
- ✔ Review projects
- ✔ Mock interviews

"Learn – Practice – Build – Optimize – Repeat
That's the Cloud Engineer Way!"

⭐ Stay Consistent. Keep Building. You've Got This! 🚀